

RAG_Medical_Reranker 实验思路存档

侧重 Hard Benchmark 阶段、Feature-based ML 解释、BCE 结果、Tiny Cross Encoder 叙事定位

生成时间：2026-06-07 17:21 | 状态：BGE 结果暂缺，后续可补入。

1. 这份存档的目的

这份文档不是最终 report，而是一个实验路线存档。它记录我们从 controlled benchmark 走到 retrieval-hard benchmark 的原因、目前结果的意义，以及后续是否加入 Tiny Cross Encoder 时应该服务什么科学问题。等最后写正式总结时，可以用它还原整个项目的逻辑链，避免变成“堆模型”和“堆结果”。

当前最重要的原则：模型不是越多越好。每新增一个模型，都必须回答一个明确问题：它和已有模型相比，控制了哪个变量？解释了哪个现象？对 RAG reranking 的理解有什么贡献？

2. 当前项目主线：从 pipeline sanity check 到 hard reranking

最初的 controlled benchmark 使用 data/processed/candidates.csv。每个问题有 1 个 positive、4 个同科室随机 hard negatives、5 个跨科室 random negatives。这个版本的意义是证明完整 pipeline 可以运行：candidate construction、embedding baseline、lightweight ML、BCE、统一 evaluate 都能顺利完成。

但是 controlled benchmark 的问题也非常明显：负例整体偏容易，embedding baseline 已经接近满分。因此，继续在 controlled benchmark 上补更多模型，边际价值很低。它适合作为 pipeline sanity check，不适合作为真正展示 reranker 价值的主实验。

为了解决这个问题，我们构造了 retrieval-hard benchmark：data/processed/candidates_hard.csv。它模拟真实 RAG top-k retrieval 场景，即候选不是随机错误答案，而是 embedding 模型认为和问题高度相似的候选。

3. Hard Benchmark 设计

项目	设计
文件	data/processed/candidates_hard.csv
规模	500 questions, 5000 query-candidate pairs
每个 qid	1 positive + 7 embedding_hard_negative + 2 random_negative
source_type	positive / embedding_hard_negative / random_negative
构造逻辑	用 nomic-embed-text 从全局候选池中检索高相似负例，排除自身、重复 context、相同 answer 和高 answer-overlap 情况。
实验目的	让候选之间更语义相近，使 embedding baseline 不再轻松满分，从而观察 reranker 在困难 top-k 场景中的真实价值。

Hard benchmark 生成结果符合预期：5000 rows、500 qids、每个 qid 正好 10 candidates，每个 qid 正好 1 positive；source_type 分布为 500 positive、3500 embedding_hard_negative、1000 random_negative。

4. 当前 Hard Benchmark 结果

Method	P@1	P@3	P@5	MRR	NDCG@5	Latency ms/pair	当前解释
embedding_baseline	0.886	0.968	0.986	0.9285	0.9416	0.0000	hard candidates 有效拉低 baseline，证明任务变难。
embedding_department_bonus	0.886	0.970	0.986	0.9290	0.9420	0.0000	科室 bonus 基本无效，说明困难不是简单科室不匹配。
lightweight_ml	0.998	1.000	1.000	0.9990	0.9993	0.0004	supervised feature-based reranker 极强，但需要解释其高分来源。
bce	0.884	0.988	0.998	0.9358	0.9516	0.5252	P@1 未提升，但改善 MRR/NDCG，说明 top-k robustness 更好。
bge	TBD	TBD	TBD	TBD	TBD	TBD	BGE 结果待组员补充。

这个结果已经达成 hard benchmark 的主要目标：embedding baseline 从 controlled benchmark 的接近满分下降到 0.886 P@1，说明候选集确实变难；而 lightweight ML 在 hard benchmark 上显著提升到 0.998 P@1，说明 supervised reranking 在固定 RAG candidate distribution 下非常有效。

5. 目前最需要谨慎解释的点：Lightweight ML 为什么这么高？

lightweight ML 的强结果是项目亮点，但也容易引发质疑。它不是直接读取原始 question-context 文本，而是使用 embedding_score、keyword_overlap、department_match、question_length、context_length、length_ratio 等 engineered retrieval features。因此，不能把它解释为“logistic regression 的语义理解能力强于 BCE”。

更严谨的解释是：lightweight ML 是一个 supervised feature-based reranker。它学习的是如何在当前 hard candidate distribution 下重新校准 retrieval-derived features。它的价值在于：当部署场景稳定、特征有效、数据规模有限时，小型监督模型可以极低延迟地提升 top-1 ranking。

因此，我们后续需要用 error analysis 和 feature importance 解释这个 0.998：模型到底修正了哪些错误？它主要依赖哪些特征？它是否过度依赖 embedding_score？这些分析比盲目继续加模型更重要。

6. BCE 结果的正确叙事

BCE 在 hard benchmark 上 P@1 = 0.884, 略低于 embedding baseline 的 0.886, 看上去不像“更强”。但是它的 P@3、P@5、MRR 和 NDCG@5 都优于 embedding baseline：P@3 从 0.968 提升到 0.988, P@5 从 0.986 提升到 0.998, NDCG@5 从 0.9416 提升到 0.9516。

这说明 BCE 的作用不是把 positive 稳定推到 rank 1, 而是减少严重排序错误, 把正确答案更稳定地保持在 top-k 范围内。报告中应将 BCE 解释为 zero-shot pretrained

cross-encoder：它直接读原文, 具备预训练语义能力, 但没有针对本项目的 hard benchmark 进行监督训练, 因此不一定在 top-1 上胜过 task-specific feature-based ML。

推荐表述：BCE improves top-k robustness and ranking quality, but it does not outperform the supervised feature-based ML reranker on P@1 under this benchmark.

7. Error Analysis 与 Feature Importance 的必要性

这两个分析是 hard benchmark

主线的收尾任务, 应该优先交给目前有空的组员完成。它们不是附属图表, 而是用于回答核心质疑：为什么 feature-based ML 表现如此强？这个强结果是否可信？

分析方向	要回答的问题	建议输出
Error Analysis	embedding baseline / BCE 排错但 ML 排对的样本是什么？ML 到底修正了哪些 top-1 错误？是否存在 ambiguous hard negatives？	results_hard/error_cases.csv, results_hard/error_summary.csv
Feature Importance	logistic regression 到底依赖哪些特征？是否只是复制 embedding_score？keyword_overlap、department_match、length_ratio 是否有贡献？	results_hard/ml_feature_importance.csv

如果 feature importance 显示 embedding_score 是主导特征, 但 keyword_overlap 或长度特征也有贡献, 可以解释为：模型不是替代 embedding retriever, 而是在 supervised setting 下对 retriever signal 做 calibration。如果 error analysis 能展示 baseline rank>1 但 ML rank=1 的案例, report 会更有说服力。

8. Tiny Cross Encoder 的叙事定位

Tiny Cross Encoder 不应该被设计成“又一个模型”或“要打败 BCE 的模型”。它真正的角色是：在 feature-based ML 和 BCE/BGE 之间补一个中间层, 用来解释 representation gap。

Model	是否监督训练	是否直接读原文	是否依赖 engineered features	是否预训练 Transf ormer	科学问题
Feature-based ML	是	否	是	否	retrieval features 能否被监督校准？
Tiny Cross Encoder	是	是	否	否	不给人工特征, 只给原文, 小模型能否学到 query-context matching？
BCE/BGE	否 (zero-shot)	是	否	是	预训练 cross-encoder 在 hard candidates 上的泛化如何？

因此, Tiny Cross Encoder 主要是和 Feature-based ML 对比, 而不是直接和 BCE/BGE 比输赢。它要回答的问题是：Feature-based ML 的高分, 到底主要来自 engineered retrieval features, 还是 raw question-context text 本身也能被小模型学出有效匹配模式？

如果 Tiny Cross Encoder 表现不错, 说明原始文本确实包含可学习的 relevance signal, feature-based ML 的高分不完全是 shortcut。如果 Tiny Cross Encoder 表现一般, 也有意义：说明在小数据医疗 reranking 场景中, 从零训练文本交互模型不如使用 retrieval-derived features 稳定, feature-based ML 是更现实的工程方案。

9. Tiny Cross Encoder 推荐训练逻辑

如果决定实施，推荐做一个 course-scale small transformer cross-encoder，而不是 fine-tune BCE/BGE。它的设计应尽量小、可解释、可复现。

输入：每一行 candidates_hard.csv 是一个样本，使用 question + [SEP] + candidate_context，label 为 0/1。训练/测试必须按 qid split，不能按 row split，避免同一个问题的 candidates 同时出现在训练和测试中。

结构建议：char-level tokenizer；[CLS] + question chars + [SEP] + context chars + [SEP]；token embedding + position embedding + segment embedding；2 层 TransformerEncoder；取 [CLS] hidden state；MLP 输出 relevance logit；loss 使用 BCEWithLogitsLoss，可使用 pos_weight 处理 1:9 类别不平衡。

输出格式必须与其他 reranker 保持一致：qid, context_id, score, rank, method, latency_ms。method 建议为 tiny_cross_encoder 或 tiny_transformer。输出路径建议 results_hard/tiny_cross_encoder_scores.csv 与 results_hard/tiny_cross_encoder_training_metrics.csv。

10. 当前推荐的工作顺序

第一，完成 hard benchmark 的 error analysis 和 feature importance。这是解释当前结果可信度的最低必要分析。第二，等待 BGE 组员补充 BGE hard benchmark 结果。第三，如时间允许，再加入 Tiny Cross Encoder，作为 feature-based ML 的 controlled comparison，而不是替代当前主线。

不建议现在重开复杂训练主线或 fine-tune BCE/BGE。当前主线已经足够强：hard benchmark construction + feature-based supervised ML + BCE top-k robustness + pending BGE + interpretability analysis。Tiny Cross Encoder 只有在能服务 “representation gap” 叙事时才值得加入。

11. 最终 report 可采用的核心叙事段落

We first observed that the controlled benchmark was too easy because the embedding baseline already achieved near-perfect ranking performance. To better evaluate reranking under realistic RAG conditions, we constructed a retrieval-hard benchmark with embedding-retrieved hard negatives. On this benchmark, the embedding baseline dropped substantially, confirming that the new candidate set is more challenging.

The supervised feature-based ML reranker achieved the strongest top-1 performance by learning to combine embedding similarity with keyword overlap, department match, and length-based features. However, because this model relies on engineered retrieval features rather than raw text, we interpret it as a feature calibration model rather than a general semantic understanding model.

BCE, as an off-the-shelf pretrained cross-encoder, directly scores raw question-context pairs. Although it does not improve P@1 in our current hard benchmark, it improves MRR and NDCG@5, suggesting better top-k ranking robustness. Pending BGE results will further complete the pretrained reranker comparison.

To further investigate why the feature-based ML model performs so strongly, we plan to conduct error analysis and feature importance analysis. If time allows, a Tiny Cross Encoder can be introduced as a controlled comparison against feature-based ML, testing whether a small supervised model can learn relevance directly from raw question-context text without engineered retrieval features.

12. 当前结论存档

目前 hard benchmark 已经达到最初目的：它成功使 candidate set 变难，并暴露出 embedding baseline 的局限。Feature-based ML 的强表现是项目亮点，但必须通过 interpretability 和 error analysis 来支撑。BCE 的结果不应被视为失败，而应解释为 zero-shot pretrained reranker 在 top-k robustness 上的提升。Tiny Cross Encoder 的意义不是堆模型，而是作为 Feature-based ML 的对照，检验 raw-text query-context interaction 在小数据条件下能学到多少。